Exceptions in Java

- An exception is a problem that arises during the execution of a program.
- When an **Exception** occurs the normal flow of the program is disrupted and the program/Application terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.

- Major reasons why an exception Occurs
  - Invalid user input
  - Device failure
  - Loss of network connection
  - Physical limitations (out of disk memory)
  - Code errors
  - Opening an unavailable file

- When an exception occurs within a method, **it creates an object**. This object is called the **exception object**. It contains information about the exception, such as the name and description of the exception and the state of the program when the exception occurred.

- Exceptions can be generated by the Java run-time system, or they can be manually generated by your code.

- Programming languages that do not support exception handling, errors must be checked and handled manually.

- There are many exception types available in Java: ArithmeticException, ArrayIndexOutOfBoundsException,IOException, SQLException

<u>Exception Handling Keywords</u>

1. try
2. catch
3. throw
4. throws
5. finally

1. try block

- Java **try** block is used to enclose the code that might throw an exception.
- If an exception occurs at the particular statement in the try block, the rest of the block code will not execute.

2. catch block

- The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.

- **Java try block must be followed by either catch or finally block.**

- The catch block must be used after the try block only. You can use multiple catch block with a single try block.

```
try {
  //  Block of code to try
}
catch(Exception e) {
  //  Block of code to handle errors
}
```

3 finally block

- We can also use the try block along with a finally block.
- Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

4.throw keyword

- To manually throw an exception, use the keyword throw.
- The **throw** statement is used in programming to explicitly raise or throw an exception.
- This is typically done in situations where an error or exceptional condition is detected during the execution of a program.

Syntax of throw keyword:

```
throw new exception_class("error message");
```

For example:

```
throw new ArithmeticException("dividing a number by 5 is not allowed in this program");
```

Throws Keyword

- The throws keyword is used in method declarations to indicate that the method might throw one or more specific types of exceptions

- we can throw multiple exceptions using the throws keyword

- Throws used
  - The method cannot handle the exception itself.
  - You want to delegate the responsibility of handling the exception to the calling code.
  - You want to maintain a clean and simple interface for the method without cluttering it with exception-handling details.

- **For example:** Lets say we have a method myMethod() the statements inside this method can throw either ArithmeticException or NullPointerException, in this case you can use try-catch as shown below:

```
public void myMethod()
{
  try {
    // Statements that might throw an exception
  }
  catch (ArithmeticException e) {
    // Exception handling statements
  }
  catch (NullPointerException e) {
    // Exception handling statements
  }
}
```

- But suppose you have several such methods that can cause exceptions, in that case it would be tedious to write these try-catch for each method. The code will become unnecessary long and will be less-readable.
- One way to overcome this problem is by using throws like this: declare the exceptions in the method signature using throws and handle the exceptions where you are calling this method by using try-catch.

```
public void myMethod() throws ArithmeticException, NullPointerException
{
  // Statements that might throw an exception
}

public static void main(String args[]) {
  try {
    myMethod();
  }
  catch (ArithmeticException e) {
    // Exception handling statements
  }
  catch (NullPointerException e) {
    // Exception handling statements
  }
}
```

throw keyword is declared inside a method body.	throws keyword is used with method signature (method declaration).
We cannot throw multiple exceptions using throw keyword.	We can declare multiple exceptions (separated by commas) using throws keyword.

Multiple catch blocks

- multiple exception handling
- When statements in a single try block generate multiple exceptions, we require multiple catch blocks to handle different types of exceptions.
- Each catch block is capable of catching a different exception. That is each catch block must contain a different exception handler.

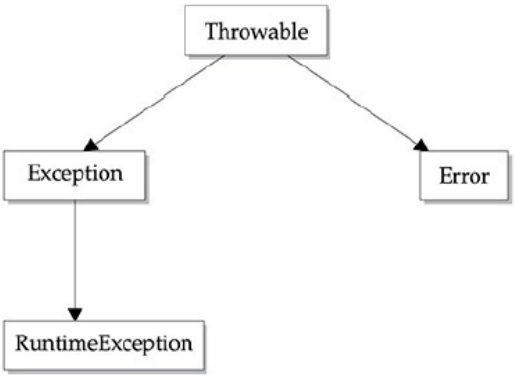
Syntax:

```
try
{
  statements;
}
catch(ExceptionType1 e1)
{
  statements;
}
catch(ExceptionType2 e2)
{
  statements;
}
catch(ExceptionType3 e3)
{
  statements;
}
  . . .
catch(ExceptionTypen en)
{
  statements;
}
```

- In the above syntax, if the exception object (ExceptionType1) is thrown by try block, the first catch block will catch the exception object thrown and the remaining catch block will be skipped.
- In case, exception object thrown does not match with the first catch block, the second catch block will check, and so on.

- At a time only one exception occurs and at a time only one catch block is used.

Exception Hierarchy



- The hierarchy of Exceptions in the Java programming language begins with the **Throwable class** – which comes from the Object class
- The Throwable class is at the top of the exception hierarchy following which are the **subclasses-Error and Exception**.
- Error and Exception should not be confused as same because there is a huge difference between these two classes.
- **Exception** is an unwanted event in a program that can be handled within the program the **error** cannot be handled in a program as it is a much serious condition that causes damage to the architecture.

- Examples of exceptions:
  IllegalArgumentException, ClassNotFoundException, NullPointerException, division by zero , invalid array indexing etc

- An **error** generally represents an unusual problem or situation from which it is difficult to recover. It does not occur by programmer mistakes. It generally occurs if the system is not working properly or resource is not allocated properly.

- Examples of error
  VirtualMachineError,StackOverFlowError,LinkageError, OutOfMmeoryError, etc

Uncaught Exceptions

- In java, assume that, if we do not handle the exceptions in a program. In this case, when an exception occurs in a particular function, then Java prints a exception message with the help of uncaught exception handler.
- When an uncaught exception occurs, the JVM calls a special private method known **dispatchUncaughtException( )**

Built-in Exceptions

- Inside the package java.lang java defines several exception classes.
- Two types of built-in exceptions
  1. Checked exceptions
  2. Unchecked exceptions

1. Checked exceptions

- Checked Exceptions are checked by the compiler at compile time.
- These checked exceptions are implicitly handled using throws clause.

Example

IOException

ClassNotFoundException

InterruptedException

FileNotFoundException etc

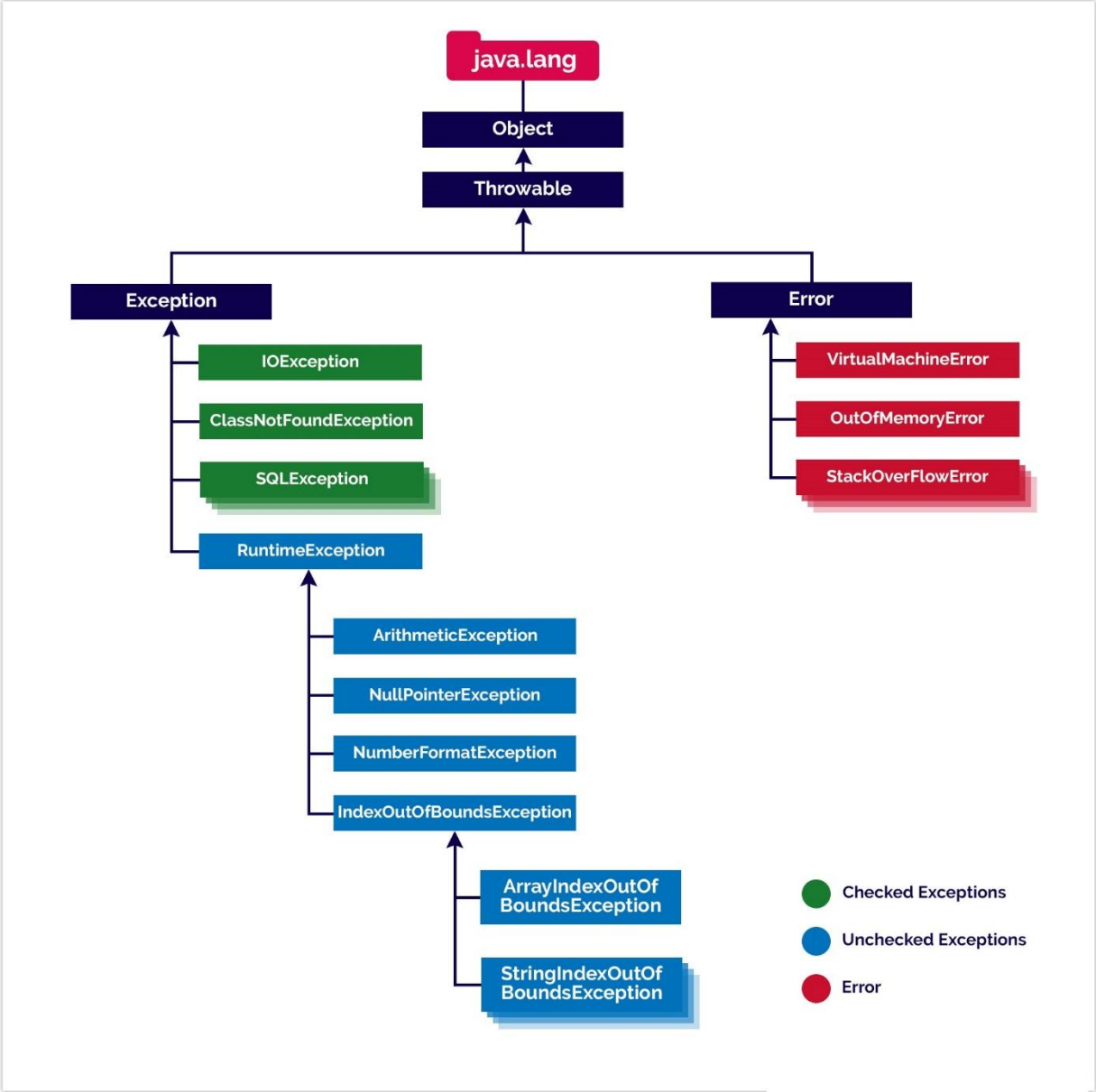
2. Unchecked Exceptions

- Unchecked Exceptions are not checked at compile time.
- Unchecked Exceptions are runtime exceptions that should be handled using try, catch or throw clause.

  Example – ArrayIndexOutOfBoundsException, ArithemeticException etc.

Unchecked Exception Example

```
class Example {
   public static void main(String args[])
   {
    int num1=10;
    int num2=0;
    int res=num1/num2;
    System.out.println(res);
   }
}
```

- If you compile this code, it would compile successfully however when you will run it, it would throw ArithmeticException. That clearly shows that unchecked exceptions are not checked at compile-time, they occurs at runtime.

java exception hierarchy